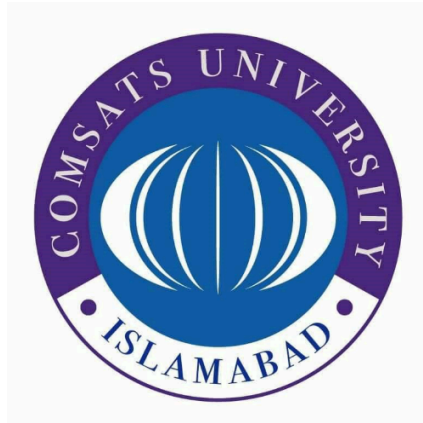




PARALLEL & DISTRIBUTED COMPUTING

LAB ASSIGNMENT #1

PREPARED BY

M Mukaram Awan
SP22-BCS-141

SUBMITTED TO

Sir Akhzar Nazeer

Problem 1: CUDA Hello World

```
#include <stdio.h>

// Kernel: prints thread ID
__global__ void sayHello() {
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    printf("Hello from thread %d\n", tid);
}

int main() {
    int threadsPerBlock = 8;
    int numBlocks = 2;

    sayHello<<<numBlocks, threadsPerBlock>>>>();
    cudaDeviceSynchronize();

    return 0;
}
```

Problem 2: Vector Addition (CPU vs GPU)

```
#include <stdio.h>
#include <cuda.h>
#include <chrono>
#include <vector>

// CUDA kernel for vector addition
__global__ void addVectorsGPU(const float *a, const float *b, float *c, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        c[idx] = a[idx] + b[idx];
    }
}
```

```

int main() {
    int n = 10000000; // 10 million elements
    size_t bytes = n * sizeof(float);

    // Allocate and initialize host memory
    std::vector<float> h_a(n, 1.0f), h_b(n, 2.0f), h_c(n);

    // Allocate device memory
    float *d_a, *d_b, *d_c;
    cudaMalloc(&d_a, bytes);
    cudaMalloc(&d_b, bytes);
    cudaMalloc(&d_c, bytes);

    // Copy inputs to device
    cudaMemcpy(d_a, h_a.data(), bytes, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, h_b.data(), bytes, cudaMemcpyHostToDevice);

    // --- CPU computation ---
    auto start_cpu = std::chrono::high_resolution_clock::now();
    for (int i = 0; i < n; i++) h_c[i] = h_a[i] + h_b[i];
    auto end_cpu = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> cpu_time = end_cpu - start_cpu;

    // --- GPU computation ---
    int threadsPerBlock = 256;
    int blocks = (n + threadsPerBlock - 1) / threadsPerBlock;

    auto start_gpu = std::chrono::high_resolution_clock::now();
    addVectorsGPU<<<blocks, threadsPerBlock>>>(d_a, d_b, d_c, n);
    cudaDeviceSynchronize();
    auto end_gpu = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> gpu_time = end_gpu - start_gpu;
}

```

```

// Copy result back
cudaMemcpy(h_c.data(), d_c, bytes, cudaMemcpyDeviceToHost);

// --- Report ---
printf("CPU time: %.6f seconds\n", cpu_time.count());
printf("GPU time: %.6f seconds\n", gpu_time.count());
printf("Speedup: %.2fx\n", cpu_time.count() / gpu_time.count());

cudaFree(d_a);
cudaFree(d_b);
cudaFree(d_c);

return 0;
}

```

problem 3: Image Inversion(CPU vs GPU)

```

#include <stdio.h>
#include <cuda.h>
#include <opencv2/opencv.hpp>
#include <chrono>

// CUDA kernel for pixel inversion
__global__ void invertImage(unsigned char* d_img, int totalPixels) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < totalPixels) {
        d_img[idx] = 255 - d_img[idx];
    }
}

int main() {

```

```

// Load grayscale image
cv::Mat img = cv::imread("input.jpg", cv::IMREAD_GRAYSCALE);
if (img.empty()) {
    printf("Could not open input image!\n");
    return -1;
}

int totalPixels = img.rows * img.cols;
unsigned char *d_img;

cudaMalloc(&d_img, totalPixels);
cudaMemcpy(d_img, img.data, totalPixels, cudaMemcpyHostToDevice);

// --- CPU inversion ---
auto start_cpu = std::chrono::high_resolution_clock::now();
cv::Mat cpu_out = img.clone();
for (int i = 0; i < totalPixels; i++)
    cpu_out.data[i] = 255 - img.data[i];
auto end_cpu = std::chrono::high_resolution_clock::now();
std::chrono::duration<double> cpu_time = end_cpu - start_cpu;

// --- GPU inversion ---
int threadsPerBlock = 256;
int blocks = (totalPixels + threadsPerBlock - 1) / threadsPerBlock;

auto start_gpu = std::chrono::high_resolution_clock::now();
invertImage<<<blocks, threadsPerBlock>>>(d_img, totalPixels);
cudaDeviceSynchronize();
auto end_gpu = std::chrono::high_resolution_clock::now();
std::chrono::duration<double> gpu_time = end_gpu - start_gpu;

cv::Mat gpu_out = img.clone();
cudaMemcpy(gpu_out.data, d_img, totalPixels, cudaMemcpyDeviceToHost);

```

```
// Save results
cv::imwrite("cpu_result.jpg", cpu_out);
cv::imwrite("gpu_result.jpg", gpu_out);

// Report
printf("CPU time: %.6f seconds\n", cpu_time.count());
printf("GPU time: %.6f seconds\n", gpu_time.count());
printf("Speedup: %.2fx\n", cpu_time.count() / gpu_time.count());

cudaFree(d_img);
return 0;
}
```